

ITech

Memoria de Proyecto PIM

Desarrollo de Aplicaciones Multiplataforma - 2º DAM

Alumno: Pau Collado Navarro

Centro: IES Salvador gadea

Curso: 2025-2026

Repositorio: <https://github.com/paucolnav2/ITech>

Índice

1. Presentación del Proyecto.....	3
1.1 ¿Qué es ITech y qué problema resuelve?.....	3
1.2 Arquitectura general.....	3
1.3 Flujo principal de datos.....	4
2. Evidencias.....	5
2.1 (Introducción a la nube pública) RA 3 - Arquitectura AWS.....	5
2.2 (Sistemas de gestión) RA 3 – Integración con Odoo ERP.....	6
2.3 (Programación de servicios y procesos) RA 3 – Pool de hilos concurrente.....	8
2.4 (Acceso a datos) RA 2 - DAOs con JDBC.....	11
2.5 (Programación multimedia y dispositivos móviles) RA 2 — Aplicación React Native.....	13
2.6 Ejecución estandarizada — Docker Compose.....	17
2.7 (Entornos de desarrollo) RA 4 — Repositorio y control de versiones.....	19
3. Reflexión Final.....	20
Organización del trabajo.....	20
Dificultades encontradas.....	20
Conclusión.....	20

1. Presentación del Proyecto

1.1 ¿Qué es ITech y qué problema resuelve?

ITech es una plataforma de monitorización industrial en tiempo real. Su objetivo es ingestar continuamente los datos de sensores industriales que recogen distintos tipos de datos (temperatura, humedad, vibración, presión, cantidad de luz o sonido), detectar situaciones de riesgo y generar automáticamente órdenes de mantenimiento correctivo, eliminando la dependencia de inspecciones manuales y reduciendo el tiempo de respuesta ante fallos.

El problema que resuelve es simple: en una instalación industrial sin monitorización centralizada, un operario no puede saber si una máquina está en estado crítico hasta que falla físicamente. Con ITech, los avisos tardan milisegundos. Cuando un sensor supera un umbral peligroso de forma sostenida, el sistema crea un ticket de intervención en el ERP sin intervención humana y marca la máquina como fuera de servicio en los paneles de control.

1.2 Arquitectura general

El sistema se compone de cinco capas independientes que se comunican entre sí:

- **Sensores industriales** → dispositivos que envían lecturas periódicas por TCP al servidor Java.
- **Servidor Java** → núcleo del sistema. Recibe conexiones concurrentes, formatea y comprueba la validez del formato de los mensajes, persiste los datos en MySQL y evalúa si el dato es una anomalía.
- **Base de datos MySQL** → almacena el catálogo de fábricas, máquinas y sensores, el histórico de lecturas y el registro de anomalías.
- **Odoo ERP** → recibe llamadas del servidor Java vía JSON-RPC para crear tickets de mantenimiento correctivo cuando se detecta una anomalía crítica.
- **Aplicaciones frontend** → ITechApp (React Native / Expo) para técnicos en planta e ITechWeb (Expo web) para supervisión general, ambas consumiendo la API HTTP del servidor Java.

1.3 Flujo principal de datos

Cuando un sensor envía una lectura, el recorrido completo es el siguiente:

1. El `ServerSocket` en `Main.java` acepta la conexión y la encola en `ConnectionQueue` haciendo uso de sockets.
2. Un hilo `ClientHandler` libre toma la conexión. Lee la primera línea y determina si es una petición HTTP (frontend) o un mensaje TCP (sensor).
3. `TCPHandler` parsea el mensaje con `GeneralParser`, extrae el ID de sensor, tipo y valor.
4. `SensorDataDAO.saveSensorData()` inserta el registro en MySQL usando un `PreparedStatement`. Durante la inserción evalúa si el valor supera el umbral definido para ese tipo de sensor.
5. Si es anomalía, `AnomalyHandler.handleAnomaly()` comprueba si se han acumulado suficientes anomalías consecutivas en el intervalo configurado. Si se supera el umbral, `MachineDAO.updateMachineState()` marca la máquina en rojo y `OdooClient.createTicket()` abre un ticket de intervención en Odoo.
6. Las apps frontend consultan periódicamente (cada N segundos) el endpoint HTTP del servidor y muestran el estado actualizado.

2. Evidencias

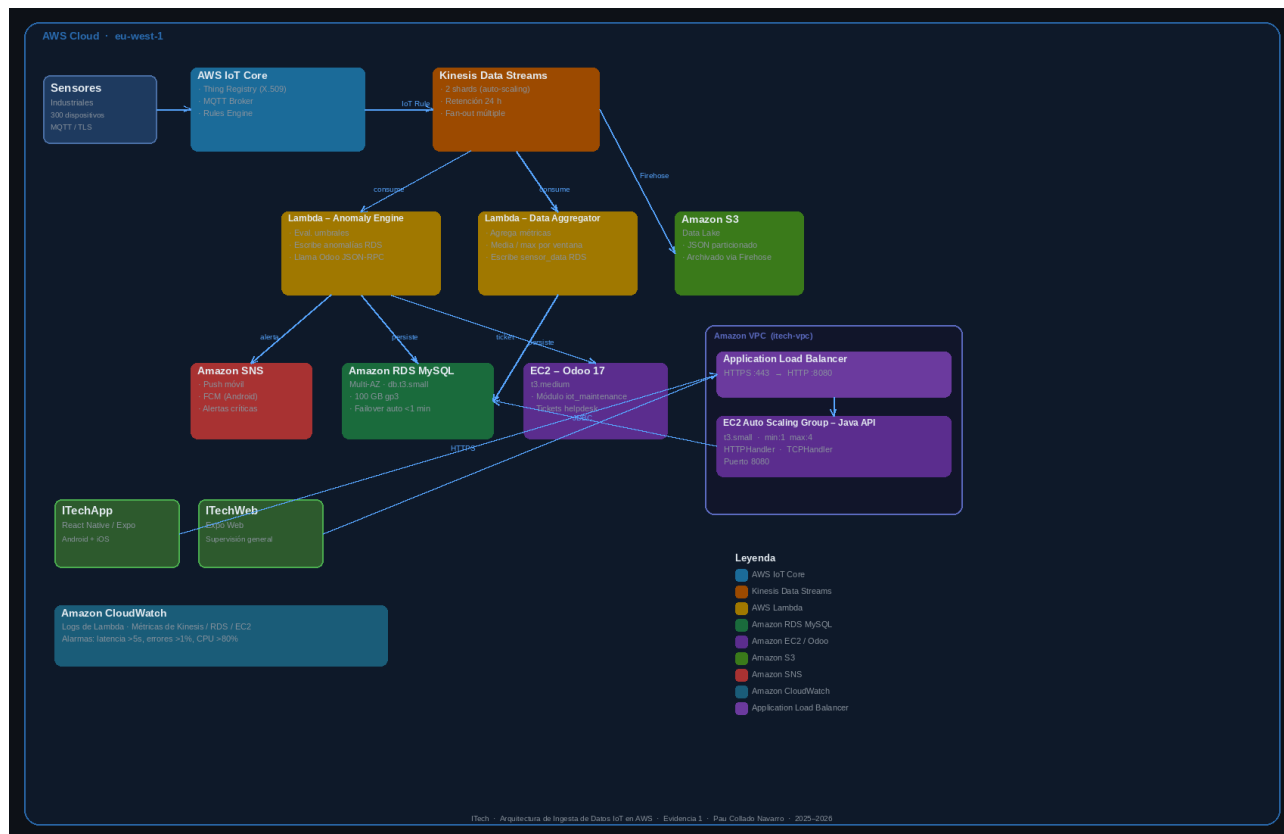
2.1 (Introducción a la nube pública) RA 3 - Arquitectura AWS

Compromiso adquirido: Diseño técnico y diagrama de arquitectura para una infraestructura de ingesta de datos en AWS utilizando servicios como AWS IoT Core, Kinesis y EC2, incluyendo tabla de costes mensuales estimados.

2.1.1 Supuestos de escala

Parámetro	Valor asumido
Número de fábricas	5
Máquinas por fábrica	20 (100 total)
Sensores por máquina	3 (300 total)
Frecuencia de envío	1 mensaje cada 10 s por sensor
Mensajes/minuto	1.800
Mensajes/día	~2.592.000
Mensajes/mes	~77.760.000
Tamaño medio de mensaje	256 bytes
Tráfico de ingesta/mes	~19,9 GB

2.1.2 Diagrama de arquitectura



2.1.3 Descripción de servicios

- AWS IoT Core** → Reemplaza la escucha TCP del servidor Java. Cada sensor industrial se registra como un Thing con un certificado X.509. Los dispositivos publican en el topic `itech/sensors/{factory_id}/{machine_id}/{sensor_id}` usando MQTT sobre TLS. El Rules Engine enruta los mensajes a Kinesis Data Streams sin necesidad de servidor intermediario. **Equivalencia en el proyecto:** `Main.java` → `ServerSocket.accept()` + `ConnectionQueue`.
- Amazon Kinesis Data Streams** → Buffer de alta capacidad que desacopla la ingesta de los consumidores. Con 2 shards admite hasta 2 MB/s de entrada (más que suficiente para los 300 sensores a 256 bytes cada 10 s). Si la escala crece, se añaden shards sin interrupción del servicio. **Equivalencia en el proyecto:** `ConnectionQueue.java` — cola de conexiones que alimenta el pool de hilos.

- **AWS Lambda – Anomaly Engine** → Función que consume el stream de Kinesis. Evalúa los umbrales definidos para cada tipo de sensor (igual que **AnomalyHandler.java**), persiste la lectura en RDS y, cuando detecta una anomalía crítica, escribe en la tabla anomalies y llama a la API XML-RPC de Odoo para crear el ticket de mantenimiento (igual que **OdooClient.java**).
- **AWS Lambda - Data Aggregator** → Segunda función consumidora del stream. Calcula agregaciones por ventanas de tiempo (media móvil de 1 minuto, máximos y mínimos del turno) y las guarda en RDS para su consulta desde las aplicaciones frontend. **Equivalencia en el proyecto:** **SensorDataDAO.java** → métodos de inserción y consulta de históricos.
- **Amazon RDS MySQL (Multi-AZ)** → Base de datos administrada con la misma estructura de la Evidencia 4 (database_script.sql). La configuración Multi-AZ garantiza failover automático con RPO < 1 minuto. Instancia db.t3.small con 100 GB de almacenamiento gp3.
- **Amazon EC2 - Java HTTP API (Auto Scaling Group)** → El servidor HTTP Java (**HTTPHandler.java**) se empaqueta como imagen **Docker** y se despliega en un Auto Scaling Group con un mínimo de 1 instancia y un máximo de 4. El Application Load Balancer distribuye el tráfico HTTPS desde las apps ITechApp e ItechWeb.
- **Amazon EC2 - Odoo ERP** → Instancia dedicada t3.medium con Odoo 17 y el módulo **iot_maintenance**. Recibe las llamadas XML-RPC desde Lambda/Anomaly Engine para crear los tickets de mantenimiento correctivo.
- **Amazon S3 - Data Lake** → Los mensajes raw de Kinesis se archivan también en S3 (vía Kinesis Firehose) en formato JSON particionado por fecha. Permite análisis histórico posterior con Amazon Athena sin impactar la base de datos operacional.
- **Amazon SNS** → Cuando Anomaly Engine detecta una alerta crítica, publica en un topic SNS. Los técnicos suscritos reciben notificaciones push en ITechApp vía AWS SNS Mobile Push (FCM para Android). **Equivalencia en el proyecto:** actualmente las alertas se consultan en la pantalla /alertas de la app vía polling HTTP.

- **Amazon CloudWatch** → Recopila logs de Lambda, métricas de Kinesis, RDS y EC2. Se configuran alarmas para: latencia del stream > 5 s, tasa de errores de Lambda > 1%, CPU de RDS > 80%.

2.1.4 Tabla de Costes Mensuales Estimados

Escenario: 300 sensores, 77,76 millones de mensajes/mes.

Servicio	Configuración	Unidad de precio	Consumo estimado/mes	Coste estimado (USD/mes)
AWS IoT Core — Conectividad	300 devices × 43.800 min	\$0,08 / millón min-conexión	13.140.000 min	\$1,05
AWS IoT Core — Mensajería	77,76 M mensajes	\$1,00 / millón mensajes	77,76 M	\$77,76
AWS IoT Core — Rules Engine	77,76 M reglas	\$0,15 / millón ejecuciones	77,76 M	\$11,66
Kinesis Data Streams — Shard hours	2 shards × 720 h	\$0,015 / shard-hora	1.440 shard-h	\$21,60
Kinesis Data Streams — PUT	77,76 M unidades	\$0,014 / millón PUT	77,76 M	\$1,09
AWS Lambda — Invocaciones	77,76 M llamadas	\$0,20 / millón (tras 1M free)	76,76 M	\$15,35
AWS Lambda — Duración	200 ms avg, 256 MB	\$0,0000166667 / GB-s	3.888.000 GB-s	\$64,80
Amazon RDS MySQL — Instancia	db.t3.small Multi-AZ	\$0,034 / hora	720 h	\$24,48
Amazon RDS MySQL — Almacenamiento	100 GB gp3 Multi-AZ	\$0,23 / GB-mes	100 GB	\$23,00
Amazon EC2 — Java API (×2 t3.small)	2 × t3.small	\$0,0208 / hora	1.440 h	\$29,95
Amazon EC2 — Odoo (t3.medium)	1 × t3.medium	\$0,0416 / hora	720 h	\$29,95
Application Load Balancer	1 ALB, ~5 LCU-h	\$0,008/hora + \$0,008/LCU-h	720 h + 3.600 LCU-h	\$34,56
Amazon S3 — Almacenamiento	20 GB/mes acumulado	\$0,023 / GB-mes	20 GB	\$0,46

Servicio	Configuración	Unidad de precio	Consumo estimado/mes	Coste estimado (USD/mes)
raw Amazon S3 — Peticiones PUT	vía Firehose	\$0,005 / 1.000 PUT	77.760 PUT	\$0,39
Amazon SNS — Notificaciones push	~10.000 alertas/mes	\$0,50 / millón	0,01 M	\$0,01
Amazon CloudWatch — Logs	~5 GB/mes	\$0,57 / GB ingestado	5 GB	\$2,85
Amazon CloudWatch — Métricas custom	20 métricas	\$0,30 / métrica/mes	20	\$6,00
Transferencia de datos salientes	apps → usuarios	\$0,09 / GB (tras 1 GB free)	~10 GB	\$0,90
			TOTAL MENSUAL	≈ \$345,86 USD

2.2 (Sistemas de gestión) RA 3 – Integración con Odoo ERP

Compromiso adquirido: Código fuente en el backend Java que envíe peticiones a la API de Odoo en el momento en que se detecte una anomalía crítica, para generar automáticamente un ticket de intervención.

La integración se implementa en dos clases que trabajan juntas: **OdooClient.java** encapsula toda la comunicación con la API de Odoo, y **AnomalyHandler.java** decide cuándo invocarla.

OdooClient utiliza la API JSON-RPC de Odoo (endpoint /jsonrpc). Al arrancar el servidor, **getOdooUID()** realiza un login con el servicio common para obtener el UID de sesión, que se cachea estáticamente para no autenticarse en cada llamada. Cuando se detecta una anomalía, **createTicket()** llama al servicio object con el método **execute_kw** sobre el modelo **helpdesk.ticket**, pasando el nombre y descripción del incidente.

AnomalyHandler.handleAnomaly() aplica una regla configurable: si **MULTIPLE_ANOMALY_REPORTING_AMOUNT** es 1, crea el ticket en el primer dato anómalo; si es mayor, **checkMultipleAnomaly()** consulta la base de datos para verificar que hay al menos N anomalías del mismo sensor en el último intervalo de tiempo antes de abrir el ticket. Esto evita falsos positivos por lecturas puntuales.

En paralelo al sistema Java, el módulo Odoo personalizado **iot_maintenance** define el modelo **helpdesk.ticket** con los campos necesarios y las vistas de gestión dentro del ERP.

Dónde verificarlo:

src/main/java/com/itech/odoo/OdooClient.java → **login()** y **createTicket()**

```
private static Integer login() {
    try {
        JsonObject body = new JsonObject();

        body.addProperty("jsonrpc", "2.0");
        body.addProperty("method", "call");

        JsonObject params = new JsonObject();
        params.addProperty("service", "common");
        params.addProperty("method", "login");
    }
}
```

src/main/java/com/itech/database/anomalyHandling/AnomalyHandler.java →
handleAnomaly(), **checkMultipleAnomaly()**

```
public class AnomalyHandler {

    public static void handleAnomaly(String dataUnit, String sensorType, Integer sensorId, double readingValue) {
        if (ConfigLoader.getMultipleAnomalyReportingAmount() == 1) {
            alertAnomaly(sensorId, sensorType, dataUnit, readingValue);
        }
        else if (checkMultipleAnomaly(dataUnit, sensorId)) {
            alertAnomaly(sensorId, sensorType, dataUnit, readingValue);
        }
    }

    public static void alertAnomaly(Integer sensorId, String sensorType, String dataUnit, double readingValue) {
        Machine machine = MachineDAO.getMachineBySensorId(sensorId);
        Factory factory = FactoryDAO.getFactoryByMachineId(machine.getFactoryId());

        MachineDAO.updateMachineState(machine.getId(), isGreen: false);

        Integer UID = OdooClient.getOdooUID();

        String name = "Anomaly detected in machine "+machine.getId()+" named \""+machine.getName()+"\".";
        String description = "Sensor "+sensorId+" which measures "+ sensorType +" detected "+readingValue+" "+dataUnit;

        OdooClient.createTicket(UID, name, description);
    }
}
```

odoo/iot_maintenance/ → módulo Odoo personalizado

IoT Maintenance Tickets						
New Maintenance Tickets						
☐ Subject	Priority	Status	Assigned To	Detected On	Closed On	
☐ Anomaly detected in machine 34 named "Machine_4_4".	★★	Resolved	Administrator	May 15, 8:19 PM	May 15, 8:34 PM	
☐ Anomaly detected in machine 64 named "Machine_7_4".	★★	New		May 13, 11:55 PM		
☐ Anomaly detected in machine 14 named "Machine_2_4".	★★	New		May 13, 11:57 PM		
☐ Anomaly detected in machine 229 named "Machine_23_9".	★★	New		May 15, 7:52 PM		
☐ Anomaly detected in machine 79 named "Machine_8_9".	★★	New		May 15, 8:36 PM		
☐ Anomaly detected in machine 219 named "Machine_22_9".	★★	New		May 15, 8:16 PM		
☐ Anomaly detected in machine 92 named "Machine_10_2".	★★	New		May 15, 8:20 PM		
☐ Anomaly detected in machine 33 named "Machine_4_3".	★★	New		May 15, 8:16 PM		
☐ Anomaly detected in machine 220 named "Machine_22_10".	★★	New		May 15, 8:17 PM		
☐ Anomaly detected in machine 63 named "Machine_7_3".	★★	New		May 15, 8:17 PM		
☐ Anomaly detected in machine 180 named "Machine_18_10".	★★	New		May 15, 8:18 PM		
☐ Anomaly detected in machine 5 named "Machine_1_5".	★★	New		May 15, 8:21 PM		

odoo/iot_maintenance/models/helpdesk_ticket.py → definición del modelo

```
from odoo import models, fields, api
from odoo.exceptions import UserError

class HelpdeskTicket(models.Model):
    # The _name must match exactly what the Java OdooClient sends: "helpdesk.ticket"
    _name = 'helpdesk.ticket'
    _description = 'IoT Maintenance Intervention Ticket'
    _inherit = ['mail.thread', 'mail.activity.mixin']
    _order = 'create_date desc'

    name = fields.Char(string='Subject', required=True, tracking=True)
    description = fields.Text(string='Description')

    state = fields.Selection(
        selection=[
            ('new', 'New'),
            ('opened', 'Opened'),
            ('assigned', 'Assigned'),
            ('in_progress', 'In Progress'),
            ('resolved', 'Resolved'),
            ('cancelled', 'Cancelled'),
        ],
        string='Status',
        default='new',
```

2.3 (Programación de servicios y procesos) RA 3 – Pool de hilos concurrente

Compromiso adquirido: Código fuente en el backend Java implementando un pool de hilos que reciba, parsee y procese de forma simultánea e independiente los flujos de datos masivos de los distintos sensores.

La concurrencia se implementa mediante una lista de espera de Java Sockets con **ConnectionQueue** y **ClientHandler**.

ConnectionQueue mantiene una **Queue<Socket>** con capacidad máxima configurable. El método **putInQueue()** es **synchronized**: si la cola está llena, el hilo principal llama a **wait()** hasta que un consumidor libere espacio. De la misma manera, **getFromQueue()** bloquea al consumidor con **wait()** si la cola está vacía, y usa **notifyAll()** para despertar a todos los hilos al modificar el estado.

Al construirse, **ConnectionQueue** lanza tantos hilos **ClientHandler** como indique la configuración. Cada **ClientHandler** extiende **Thread** y ejecuta un bucle infinito: extrae un socket de la cola, lee la primera línea del mensaje y decide el tipo de conexión. Si empieza por **GET**, **POST**, **PUT** o **DELETE**, delega en **HTTPHandler** (petición de una app frontend); en cualquier otro caso, delega en **TCPHandler** (mensaje de sensor industrial). Esta distinción permite que el mismo pool atienda simultáneamente sensores y aplicaciones cliente.

Dónde verificarlo:

src/main/java/com/itech/ConnectionQueue.java → cola con wait/notifyAll

```
public class ConnectionQueue {
    private Queue<Socket> queue = new LinkedList<>();
    private final int maxSize;

    public ConnectionQueue(int maxSize) {
        this.maxSize = maxSize;
        for (int i = 0; i < maxSize; i++) {
            new ClientHandler(queue, this).start();
        }
    }

    public synchronized void putInQueue(Socket socket) {
        while (queue.size() >= maxSize) {
            try {
                wait();
            } catch (InterruptedException e) {
                Thread.currentThread().interrupt();
                throw new RuntimeException("Couldn't put in queue: " + e.getMessage());
            }
        }
        queue.add(socket);
        notifyAll();
    }

    public synchronized Socket getFromQueue() {
        while (queue.isEmpty()) {

```

src/main/java/com/itech/clientHandlers/ClientHandler.java → detección HTTP vs TCP, bucle de worker

```
public class ClientHandler extends Thread {
    private final ConnectionQueue queue;

    public ClientHandler(com.itech.ConnectionQueue queue) { this.queue = queue; }

    @Override
    public void run() {
        while (true) {
            Socket socket = queue.getFromQueue();
            if (socket != null) {
                try {
                    BufferedReader input = new BufferedReader(new InputStreamReader(socket.getInputStream()));
                    BufferedWriter output = new BufferedWriter(new OutputStreamWriter(socket.getOutputStream()));
                } {
                    String firstLine = input.readLine();

```

TCPHandler.java → formateo y persistencia de lecturas de sensor

```
package com.itech.clientHandlers;

import ...

public class TCPHandler {
    private BufferedReader input;
    private BufferedWriter output;

    public TCPHandler(BufferedReader input, BufferedWriter output) {
        this.input = input;
        this.output = output;
    }

    //FORMATO "SENSORID;TEMPERATURE;86"
    public void handle (String firstLine) {
        String[] sensorMessage = Validator.validateSensorMessage(firstLine);

        SensorDataDAO.saveSensorData(sensorMessage);
    }
}
```

HTTPHandler.java → API REST (manual) para el frontend

```
public class HTTPHandler {
    private BufferedReader input;
    private BufferedWriter output;

    public HTTPHandler(BufferedReader input, BufferedWriter output) {
        this.input = input;
        this.output = output;
    }

    public void handle(String firstLine) {
        List<String> headerList = new ArrayList<>();
        headerList.add(firstLine);
        String line;
        try {
            while (!(line = input.readLine()).isEmpty()) {
                headerList.add(line);
            }
        } catch (IOException e) {
            throw new RuntimeException("Couldn't read http message: " + e.getMessage());
        }

        String[] header = headerList.toArray(new String[0]);
        String method = HttpParser.getMethod(header);
        String route = HttpParser.getRoute(header);

        if (method.equals("OPTIONS")) {

```

2.4 (Acceso a datos) RA 2 - DAOs con JDBC

Compromiso adquirido: Clases DAO en Java utilizando JDBC que ejecuten operaciones CRUD sobre la base de datos relacional, archivo .sql exportado con la estructura y script que genere un lote de datos.

La capa de acceso a datos sigue el patrón DAO (Data Access Object): cada entidad del dominio tiene su propia clase con métodos estáticos que construyen y ejecutan **PreparedStatement** contra MySQL. El uso de **PreparedStatement** en lugar de concatenación de strings previene inyección SQL en todos los puntos de entrada. **DatabaseManager** centraliza la gestión de conexiones mediante un pool, devolviendo una **Connection** lista con **getConnection()**.

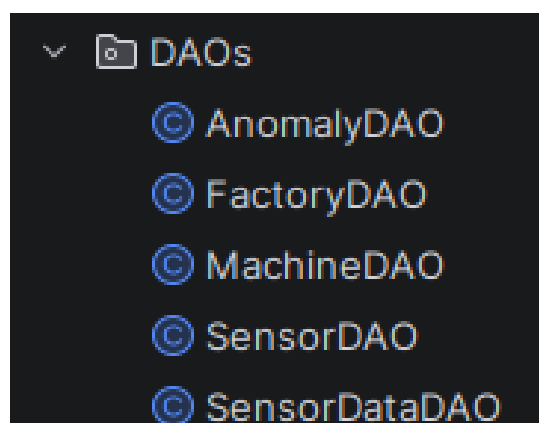
Los cinco DAOs son:

- **FactoryDAO** — consulta fábricas por ID y por máquina.
- **MachineDAO** — consulta máquinas, actualiza su estado (hasGreenState) y verifica si tienen anomalías recientes.
- **SensorDAO** — valida que un sensor pertenece a una máquina y que su tipo es coherente.
- **SensorDataDAO** — inserta lecturas, evalúa el umbral de anomalía durante la inserción y recupera el histórico por sensor.
- **AnomalyDAO** — registra anomalías con su timestamp, tipo y valor.

El archivo **database_script.sql** contiene el DDL completo del esquema (tablas factories, machines, sensors, sensor_data, anomalies, data_units) y un bloque de datos de prueba con fábricas, máquinas y sensores de ejemplo.

Dónde verificarlo:

src/main/java/com/itech/database/DAOs/ → los cinco DAOs



src/main/java/com/itech/database/DatabaseManager.java → gestión del pool de conexiones

```
package com.itech.database;

import ...

public class DatabaseManager {
    private static final String URL = "jdbc:mysql://" + ConfigLoader.getDatabaseIPDirection() + ":"
        + ConfigLoader.getDatabasePort() + "/" + ConfigLoader.getDatabaseName();
    private static final String USER = ConfigLoader.getDatabaseUsername();
    private static final String PASSWORD = ConfigLoader.getDatabasePassword();

    public static void validateDatabaseConnection() {
        try {
            DriverManager.getConnection(URL, USER, PASSWORD);
        } catch (SQLException e) {
            Throwable cause = e.getCause();
            throw new FailedConnectionToDatabase(Objects.requireNonNullElse(cause, e).getMessage());
        }
    }
}
```

src/main/java/com/itech/database/database_script.sql → esquema DDL + datos de prueba

```
CREATE DATABASE IF NOT EXISTS ITech;

USE ITech;

DROP TABLE IF EXISTS sensor_data;
DROP TABLE IF EXISTS sensor_unit_types;
DROP TABLE IF EXISTS sensor;
DROP TABLE IF EXISTS machine;
DROP TABLE IF EXISTS factory;
DROP TABLE IF EXISTS data_units;
DROP TABLE IF EXISTS unit_types;
DROP PROCEDURE IF EXISTS populate_data;

CREATE TABLE unit_types(
    id SMALLINT AUTO_INCREMENT PRIMARY KEY,
    name VARCHAR(50) NOT NULL UNIQUE
);

INSERT INTO unit_types(name) VALUES (name 'TEMPERATURE'), (name 'VIBRATION'), (name 'HUMIDITY'), (name 'PRES');

CREATE TABLE data_units(
    id SMALLINT AUTO_INCREMENT PRIMARY KEY,
    name VARCHAR(100) NOT NULL UNIQUE,
    unit_type_id SMALLINT NOT NULL,
    CONSTRAINT fk_unit_type FOREIGN KEY (unit_type_id) REFERENCES unit_types(id) ON DELETE CASCADE
);
```


2.5 (Programación multimedia y dispositivos móviles) RA 2 — Aplicación React Native

Compromiso adquirido: Código fuente de la aplicación móvil en React Native con dashboard general, pantallas de detalle por máquina y sistema de alertas.

ITechApp está desarrollada con **React Native** + **Expo** y usa **Expo Router** para la navegación, con una estructura de tres niveles: un stack raíz que contiene un drawer lateral, que a su vez incluye un tab navigator para las pantallas principales.

Las tres pantallas del tab navigator cubren los requisitos de la evidencia:

- **Dashboard** (/dashboard) — muestra un resumen de todas las fábricas con el número de máquinas activas e incidentes.
- **Sensores** (/sensores) — listado de sensores con su última lectura y unidad.
- **Alertas** (/alertas) — registro de anomalías recientes con timestamp, tipo de sensor, valor detectado y máquina afectada.

Fuera del tab navigator, hay pantallas de detalle accesibles desde el drawer o por navegación: /fabricas lista todas las fábricas, y desde ahí se puede navegar a /screens/fabrica, /screens/maquina y /screens/sensor para ver el estado y el histórico de lecturas de cada elemento.

Cada pantalla consume un hook personalizado ([useFactories](#), [useMachines](#), [useSensors](#), [useSensorData](#), [useAnomalies](#)) que hace polling HTTP periódico al servidor Java y devuelve los datos tipados con sus interfaces [TypeScript](#) correspondientes.

Dónde verificarlo:

ITechApp/app/(stack)/(drawer)/(tabs)/dashboard/ → pantalla dashboard

```
import ...

const dashboard = () => {
  const { data: machines, loading: loadingMachines, error: errorMachines, refetch: refetchMachines } = useMachines();
  const { data: sensors, loading: loadingSensors, refetch: refetchSensors } = useSensors();

  const isRefreshing = loadingMachines || loadingSensors;

  const onRefresh = () => {
    refetchMachines();
    refetchSensors();
  };

  const sensorCountByMachine = useMemo(() => {
    const map: Record<number, number> = {};
    sensors.forEach((s) => {
      map[s.machineID] = (map[s.machineID] ?? 0) + 1;
    });
    return map;
  }, [sensors]);

  const stats = useMemo(() => {
    const total = machines.length;
    const ok = machines.filter((m) => m.hasGreenState).length;
    const alert = total - ok;
    return { total, ok, alert };
  }, [machines]);
}
```

ITechApp/app/(stack)/(drawer)/(tabs)/alertas/ → listado de anomalías

```
import ...

const Alertas :() => any = () :any => {
  const { data, loading, error, refetch } = useAnomalies();
  const { data: machines, refetch: refetchMachines } = useMachines();

  const alertMachines :any = useMemo(
    () :any => machines.filter((m :any) :boolean => !m.hasGreenState),
    [machines],
  );

  const onRefresh :() => void = () :void => {
    refetch();
    refetchMachines();
  };

  const listHeader :any = alertMachines.length > 0 ? (
    <View style={styles.alertMachinesSection}>
      <View style={styles.sectionTitleRow}>
        <Ionicons name="warning" size={16} color={Colors.danger} />
        <Text style={styles.sectionTitle}>
          Máquinas en alerta ({alertMachines.length})
        </Text>
      </View>
    </View>
  ) : null;
}
```

ITechApp/app/(stack)/(drawer)/(tabs)/sensores/ → listado de sensores

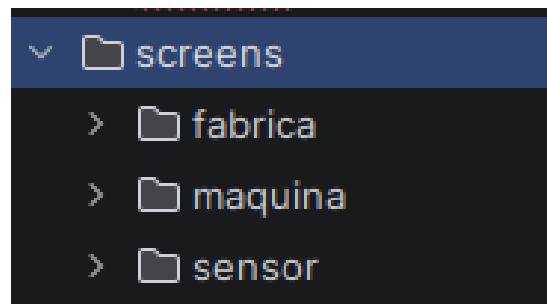
```
import ...

const Sensores = () => {
  const { data, loading, error, refetch } = useSensors();
  const { data: machines } = useMachines();
  const [query, setQuery] = useState("");

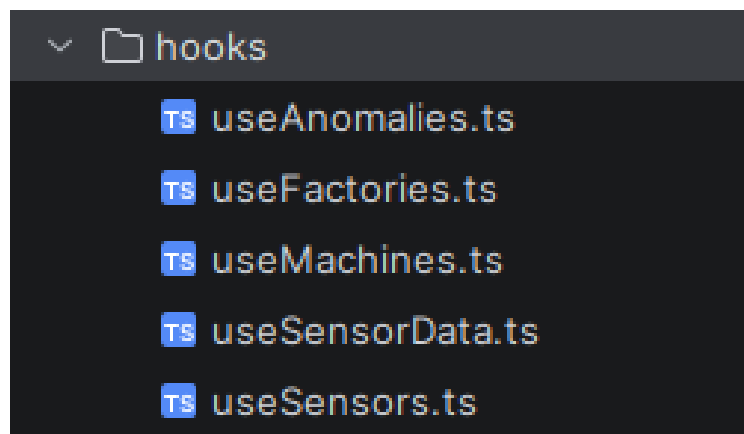
  const machineStateMap = useMemo(() => {
    const map: Record<number, boolean> = {};
    machines.forEach((m) => { map[m.id] = m.hasGreenState; });
    return map;
  }, [machines]);

  const filtered = query.trim()
    ? data.filter(
        (s) =>
          s.sensorName.toLowerCase().includes(query.toLowerCase()) ||
          s.sensorTypes.some((t) => t.toLowerCase().includes(query.toLowerCase())),
      )
    : data;
}
```

ITechApp/app/(stack)/screens/ → detalle fábrica, máquina y sensor



ITechApp/hooks/ → [useAnomalies](#), [useMachines](#), [useSensorData...](#)



ITechApp/components/MachineCard.tsx → tarjeta de estado verde/rojo

```
import ...

const MaquinaDetail :() => any = () :any => {
  const { id } = useLocalSearchParams<{ id: string }>();
  const machineId :number = Number(id);

  const { data: machine, loading: loadingMachine } = useMachine(machineId);
  const { data: machineSensors, loading: loadingSensors } = useMachineSensors(machineId);

  const { data: allSensors, loading: loadingAll } = useSensors();
  const fallbackSensors :any = useMemo(
    () :any => allSensors.filter((s :any ) :boolean => s.machineID === machineId),
    [allSensors, machineId],
  );

  const sensors :any = machineSensors.length > 0 ? machineSensors : fallbackSensors;
  const loading :any = loadingMachine || loadingSensors;

  if (loading && !machine) {
```

ITechApp/components/AnomalyCard.tsx → tarjeta de alerta

```
import ...

interface AnomalyCardProps {
  anomaly: Anomaly;
}

const AnomalyCard = ({ anomaly }: AnomalyCardProps) => {
  const typeColor = SensorTypeColors[anomaly.sensorType] ?? Colors.danger;
  const date = new Date(anomaly.dateAndTime);
  const timeStr = date.toLocaleTimeString("es-ES", { hour: "2-digit", minute: "2-digit" });
  const dateStr = date.toLocaleDateString("es-ES", { day: "2-digit", month: "2-digit" });

  return (
    <View style={[styles.card, { borderLeftColor: typeColor }]}>
      <View style={styles.row}>
        <Icons name="warning" size={18} color={Colors.danger} />
        <View style={styles.info}>
          <Text style={styles.sensorName}>{anomaly.sensorName}</Text>
          <Text style={styles.machine}>
            {anomaly.machineName} - {anomaly.factoryName}
          </Text>
        </View>
      </View>
    </View>
  );
}
```

2.6 Ejecución estandarizada — Docker Compose

Compromiso adquirido: Docker o script para levantar la base de datos, el ERP Odoo, el backend Java y un simulador de sensores de forma simple y estandarizada.

El archivo docker-compose.yml en la raíz del proyecto define cinco servicios que arrancan con un único comando:

- **Mysql** → imagen mysql:8, inicializa la base de datos ITech, con healthcheck que bloquea el arranque del servidor Java hasta que **MySQL** esté listo.
- **Postgres** → imagen postgres:15, base de datos interna de **Odoo** (requerida por **Odoo**).
- **Odoo** → imagen odoo:19, monta el directorio ./odoo como addons extra para cargar el módulo **iot_maintenance** automáticamente.
- **Java** → construida desde el **Dockerfile** del proyecto, espera a que **MySQL** y **Odoo** estén sanos antes de arrancar.
- **Simulator** — construida desde ./data_simulator, arranca tras el servidor **Java** y comienza a enviar lecturas simuladas de sensores.

Todos los servicios comparten la red interna itech_network, por lo que no se exponen puertos innecesarios al exterior salvo los de acceso (8080 para el servidor **Java**, 8069 para **Odoo**).

Dónde verificarlo:

docker-compose.yml → definición de los cinco servicios y sus dependencias

```
version: '3.8'

services:

  mysql:
    image: mysql:8
    container_name: itech_mysql
    restart: always
    environment:
      MYSQL_ROOT_PASSWORD: root
      MYSQL_DATABASE: ITech
    ports:
      - "3306:3306"
    volumes:
      - mysql_data:/var/lib/mysql
    networks:
```

Dockerfile → imagen del servidor **Java**

```
FROM maven:3.9.9-eclipse-temurin-21 AS build

WORKDIR /app

COPY config ./config

COPY pom.xml .
RUN mvn dependency:go-offline

COPY src ./src

RUN mvn clean package -DskipTests

FROM eclipse-temurin:21-jdk

WORKDIR /app

COPY --from=build /app/target/app.jar app.jar
COPY --from=build /app/config ./config

CMD ["java", "-jar", "app.jar"]
```

data_simulator/ → imagen del simulador de sensores

```
#!/bin/sh

HOST=java
PORT=8080

# — Threshold limits (mirrors config.properties limit.amount.*) —————
LIMIT_TEMPERATURE=100
LIMIT_HUMIDITY=98
LIMIT_VIBRATION=3000
LIMIT_PRESSURE=5
LIMIT_LIGHT=28000
LIMIT_SOUND=180
LIMIT_MOISTURE=70

# — Sensible physical minimums per sensor type —————
MIN_TEMPERATURE=15
MIN_HUMIDITY=20
MIN_VIBRATION=10
MIN_PRESSURE=1
MIN_LIGHT=50
MIN_SOUND=30
MIN_MOISTURE=5
```


2.7 (Entornos de desarrollo) RA 4 — Repositorio y control de versiones

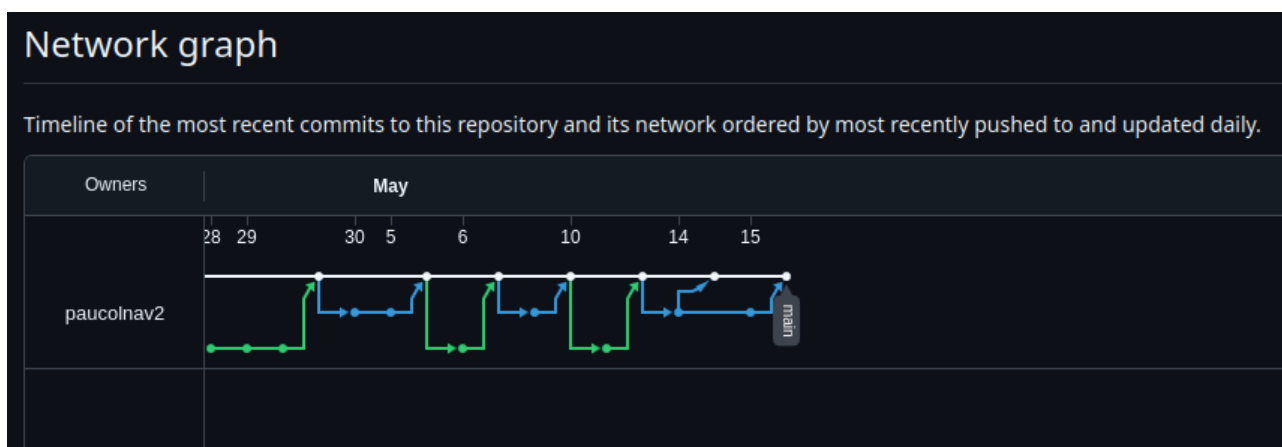
Compromiso adquirido: Repositorio con al menos dos ramas (main y feature), commits adecuados y profesionales.

El proyecto se desarrolló en GitHub siguiendo un flujo de trabajo basado en ramas de funcionalidad. La rama main contiene siempre el estado entregable del proyecto. Cada bloque de desarrollo (servidor **Java**, módulo **Odoo**, frontend móvil, **Docker**) se realizó en su propia rama feature/*, que al completarse se mergeó a main mediante Pull Request.

Los mensajes de commit siguen una convención descriptiva técnica, a veces en inglés y a veces en español, reflejando el contenido del cambio de forma concisa.

Dónde verificarlo:

github.com/paucolnav2/ITech → historial de ramas, commits y Pull Requests



3. Reflexión Final

Organización del trabajo

Primero realicé primero el servidor TCP con el pool de hilos, luego el esquema de base de datos y los DAOs, después la lógica de anomalías y la integración con Odoo, y finalmente el frontend móvil. La funcionalidad de Docker se añadió en cierto punto pero se dejó como carcasa (es decir, levantaba los contenedores pero estaban vacíos y no eran funcionales).

Dificultades encontradas

La mayor dificultad fue la gestión del tiempo, ya que mi proyecto es bastante más grande que otros proyectos PIM similares. Además de eso, he tenido que realizar el proyecto mientras hacía las prácticas en GFT Technologies, que me restaba 11 horas y media del día (9 horas y media al día más una hora de ida y vuelta). También ha sido duro implementar la funcionalidad de Odoo, ya que siempre da problemas, y cuando los solucionas, aparecen nuevos.

Conclusión

Creo que ha sido un proyecto del que he aprendido mucho, en todos los aspectos y tecnologías. Ha sido muy satisfactorio ir completando los apartados (o funcionalidades) una a una e ir viendo cómo funcionaba. Ha habido veces en las que me he tenido que contener para no implementar nuevas features que me hubieran llevado días. Muy divertido y didáctico.